
LEARNER ROADMAP

From Advanced RAG to Production AI Systems

You have built agents and advanced retrieval pipelines that work on your machine. This is the path from a working prototype to a system real users depend on.

AUDIENCE

Practitioners who have completed Agent API and Advanced RAG

FORMAT

Self-directed roadmap with build milestones

SUGGESTED PACE

12 to 16 weeks part time

ORIENTATION

Where you are, and what changes next

The hardest part of this field is not building the thing. It is making the thing behave the same way tomorrow as it did today, for someone who is not you.

By this point you can wire an agent to tools, chunk and embed a corpus, run hybrid search, rerank results, and get useful answers out of the other end. That is genuinely the harder half of the learning curve conceptually. But there is a gap between that and a system in production, and the gap is not made of more clever prompting.

What changes is the nature of the problem. Prototype work is about **capability**: can the system do this at all? Production work is about **reliability, cost, and consequence**: does it do this correctly nine times out of ten, at a price that makes sense, and does something sane the tenth time?

What you optimized for so far

- Does the answer look right?
- Does the agent call the tool?
- Does retrieval find the passage?
- Speed of iteration in a notebook

What you optimize for now

- Is it right across 200 cases, repeatably?
- What happens when the tool fails?
- What does one query cost, at scale?
- Can someone else debug it at 2am?

THE SINGLE MOST IMPORTANT SHIFT

Stop evaluating by reading outputs. Start evaluating with a dataset and a score. Almost every other practice in this roadmap depends on you being able to answer the question "did that change make it better?" with a number rather than an impression. If you take one thing from this document, take that.

How to use this roadmap

The six stages below are ordered by dependency, not by difficulty. Stage 1 makes every later stage measurable, so resist the temptation to skip it for the more exciting material in Stage 3. Each stage names what to learn, what to build, and how to know you are done.

Work on one real system throughout. Pick a use case with actual users, even a handful, even internal ones. Toy problems will not surface the failure modes this roadmap exists to teach you.

Six stages to production

Roughly two to three weeks each, part time. The order matters more than the pace.

01 Evaluation and measurement

Build the ruler before you build anything else. Datasets, graders, regression suites.

02 Reliability engineering

Failure modes, retries, fallbacks, structured outputs, guardrails, graceful degradation.

03 Agent architecture at scale

Context management, memory, multi-agent decomposition, and when not to use an agent.

04 Serving, cost, and latency

Deployment shapes, streaming, caching, model routing, and the economics of a query.

05 Observability and operations

Tracing, logging, monitoring drift, incident response, and the human feedback loop.

06 Security, safety, and governance

Prompt injection, data boundaries, access control, auditability, and compliance posture.

A note on tools versus principles

This document names some tools, but treats them as examples rather than requirements. The tooling in this space changes every few months; the underlying problems do not. Learn the problem well enough that you can evaluate any tool that claims to solve it, and you will not need to relearn the field each year.

VERIFY BEFORE YOU COMMIT

Specific model names, pricing, context limits, and library APIs referenced anywhere in this space go stale quickly. Check current provider documentation before making architectural decisions that depend on them.

Evaluation and measurement

Everything downstream depends on this. Without evals you are not engineering, you are guessing with extra steps.

The reason teams stall between prototype and production is almost always the same: they cannot tell whether a change helped. Someone tweaks a prompt, spot-checks four outputs, ships it, and quietly breaks a category of queries nobody was looking at. Evals are the fix.

What to learn

FOUNDATION

Building an evaluation dataset

Start with 30 to 50 real examples, not synthetic ones. Pull them from actual usage, support tickets, or domain experts. Cover the common path, the edge cases, and the things that should be refused. Grow to a few hundred over time. A small dataset of real cases beats a large dataset of invented ones.

CORE SKILL

Choosing graders

Different outputs need different scoring. Learn all three and know when each applies.

- **Deterministic checks** for anything verifiable: valid JSON, correct tool called, citation present, number matches source. Cheap, fast, unambiguous. Use these wherever possible.
- **LLM-as-judge** for qualitative dimensions: faithfulness to sources, tone, completeness. Requires a careful rubric and calibration against human labels, or you are measuring the judge's quirks.
- **Human review** for the ambiguous residue, and for periodically checking that your automated graders still agree with people.

RAG SPECIFIC

Separating retrieval failures from generation failures

When a RAG answer is wrong, there are two distinct causes and they need different fixes. Measure them separately.

- **Retrieval quality:** did the right chunks come back at all? Measure with `recall@k` and `precision@k` against known-correct passages.
- **Generation quality:** given correct context, did the model use it faithfully? Measure groundedness and whether claims trace to retrieved text.

Teams that skip this distinction spend weeks tuning prompts when the actual problem was chunking.

AGENT SPECIFIC

Trajectory evaluation

Agents can reach a correct answer through a wasteful or dangerous path. Score the path as well as the destination: number of steps, tool selection accuracy, unnecessary calls, recovery after a failed step, and whether it terminated rather than looping.

BUILD MILESTONE

An eval suite that runs from one command, scores your system across your dataset, and prints a comparison against the last run. Wire it into CI so no change ships unmeasured. **Done when:**

- ☐ You can state your system's accuracy as a number, per category
- ☐ A prompt change produces a score delta in minutes, not a debate
- ☐ You have caught a regression you would otherwise have shipped

Reliability engineering

Assume every component fails. Design for what happens next.

A demo runs the happy path. Production runs every path. Malformed output, a vector store timeout, a rate limit, a 400-page paste, an agent that loops. None are exotic; all will happen in your first month of real traffic.

Failure modes to design against

FAILURE	TYPICAL MITIGATION
Malformed structured output	Constrained decoding or schema-enforced output modes; validate against a schema and retry with the parse error fed back as context
Provider timeout or 5xx	Exponential backoff with jitter, bounded retries, and a circuit breaker so one bad dependency does not cascade
Rate limiting	Client-side queueing, per-tenant budgets, and a smaller model or secondary provider as overflow
Agent loops without terminating	Hard step ceilings, wall-clock and token budgets, loop detection on repeated state, forced summarize-and-exit
Tool returns garbage or errors	Validate tool output before it enters context; return a structured error the agent can reason about, not a stack trace
Retrieval returns nothing relevant	Relevance thresholds, and an explicit "no information on this" path rather than forcing an answer from weak context

Principles worth internalizing

Fail loudly to your logs, gracefully to your user

A user should see a clear, honest message; your telemetry should see the full trace. The common mistake is inverting this.

Idempotency for anything with side effects

If an agent can send an email, create a ticket, or charge a card, a retry must not do it twice. Idempotency keys and a record of executed actions are mandatory, not optional. This is the single most damaging category of agent bug in production.

Degrade in tiers, not to zero

If reranking is down, serve unranked results. If the large model is unavailable, serve the smaller one with a note. A partially useful answer beats an error page in most contexts.

DESIGN RULE

Any agent action that is irreversible, expensive, or externally visible should require either explicit confirmation or a human in the loop. Decide this per action at design time, not per incident afterwards.

BUILD MILESTONE

Deliberately break each dependency in turn: kill the vector store, force a malformed response, trigger a rate limit. Every one should produce a defined behaviour and a useful log entry. Add each scenario to your eval suite as a permanent test.

Agent architecture at scale

Context is your scarcest resource. Most agent architecture is really context architecture.

Prototype agents work because the task fits in the window and the model holds the whole problem at once. Real tasks are longer, use more tools, and accumulate history. Managing what the model sees, and when, becomes the central design activity.

Context management

- **Compaction:** summarize completed sub-tasks rather than carrying raw transcripts. Preserve decisions and outcomes; discard the deliberation.
- **Externalized state:** keep artifacts in files or a store and pass references, not contents. The agent retrieves what it needs when it needs it.
- **Tool result pruning:** large tool outputs fill a window fastest. Filter, truncate, or summarize before insertion.
- **Attention to position:** models attend unevenly across long contexts. Place critical material where it is reliably used, and verify empirically rather than assuming.

Memory

Three kinds, with different storage and retrieval needs.

TYPE	HOLDS	IMPLEMENTATION
Working	Current task state	The context window plus scratch storage; cleared per task
Episodic	What happened in past sessions	Conversation store, retrieved by relevance or recency
Semantic	Durable facts and preferences	Structured store; explicitly written, explicitly read

Resist storing everything. Unbounded memory degrades retrieval precision and quietly raises cost on every request.

Single agent or many

Multi-agent architectures are fashionable and often unnecessary. Each additional agent adds a handoff, and handoffs lose information.

Prefer a single agent when

- The task is sequential and coherent
- Full context helps at every step
- Latency matters
- You are early and still learning the problem

Consider decomposition when

- Sub-tasks are genuinely parallel
- Each needs a distinct, large toolset
- Different steps need different models or permissions
- One agent's context is provably overloaded

Three patterns worth knowing. *Orchestrator with subagents:* a lead delegates to workers with clean contexts; good for research. *Sequential pipeline:* fixed specialized stages, deterministic ordering. *Evaluator loop:* a generator produces, a critic scores, iterate to a threshold; reliable but multiplies cost.

WORTH SAYING PLAINLY

Many tasks people build agents for are better served by a deterministic pipeline with one or two model calls inside it. Agents are for problems where the sequence of steps genuinely cannot be known in advance. If you can draw the flowchart, write the flowchart.

Retrieval as a capability

The layer past hybrid search and reranking is giving the agent retrieval as a tool rather than a fixed pre-step: reformulating queries, searching iteratively, deciding when it has enough. Also study structured retrieval over graphs and databases, where relationships matter more than proximity.

BUILD MILESTONE

Refactor your agent to run a task that exceeds its context window comfortably, using compaction and externalized state. Measure against your eval suite before and after. If quality drops, your compaction is discarding the wrong things.

Serving, cost, and latency

A system nobody can afford to run is not in production. Neither is one nobody will wait for.

Deployment shape

Most of what you know about deploying services applies. The differences worth attention:

- **Long-running requests** break assumptions in typical web infrastructure. Timeouts, load balancer limits, and connection handling all need revisiting.
- **Streaming** is the main lever on perceived latency. First-token time matters more to users than total time. Design the API for it from the start; retrofitting streaming is painful.
- **Asynchronous execution** suits agent runs lasting minutes. Accept the job, return an identifier, stream or poll for progress. Never hold an HTTP connection open for a ten-minute run.
- **Statelessness** in your service layer, with conversation and agent state in an external store, keeps horizontal scaling straightforward.

Cost control

Cost is a design property, not a billing surprise. Levers in rough order of impact:

LEVER	WHAT IT DOES
Prompt caching	Reuses compute for a stable prefix across calls. Large savings where instructions or shared context are long and fixed. Put the stable part first.
Model routing	Send easy requests to a small fast model, hard ones to a large one. A classifier or heuristic front door. Often the largest single saving available.
Context discipline	Every unnecessary token in context is paid for on every call. Trimming retrieved chunks and tool output is directly a cost reduction.
Semantic caching	Serve repeat or near-repeat queries from cache. Needs care around freshness, but common in high-traffic Q&A.
Batching	For non-interactive workloads, batch processing is typically much cheaper than real-time calls.
Step budgets	Agent cost is unbounded without ceilings. Cap steps, tokens, and tool calls per task.

INSTRUMENT THIS EARLY

Track cost per request, per user, and per feature from day one. Teams that add cost tracking after launch invariably discover that a small fraction of usage patterns accounts for most of the bill, and by then it is embedded in user habits.

Latency

Budget it explicitly. Retrieval, reranking, generation, and each tool call contribute. Measure the breakdown, then attack the largest component. Parallelize independent tool calls, run retrieval concurrently with preprocessing, and accept that some tasks are simply slow, in which case the answer is progress feedback rather than optimization.

BUILD MILESTONE

Produce a cost-and-latency profile of one representative request, broken down by component. Then find a 30 percent reduction in either, without a measurable quality drop on your eval suite.

Observability and operations

You cannot fix what you cannot see. Non-deterministic systems need better telemetry than deterministic ones, not worse.

When a traditional service misbehaves you read the stack trace. When an agent misbehaves there is no stack trace, only a sequence of decisions that each looked locally reasonable. Tracing is how you reconstruct it.

What to capture

- **Full traces:** every model call, tool invocation, retrieval, and their inputs and outputs, linked by a trace identifier that spans the whole request.
- **Versioning:** which prompt version, model version, and retrieval index served this request. Without this you cannot correlate a quality change to a deploy.
- **Token and cost accounting** per span, so expensive steps are visible.
- **User feedback** attached to the trace: thumbs, corrections, abandonment, escalation to a human.

What to monitor

SIGNAL	WHY IT MATTERS
Eval scores on live traffic	Run graders on a sample of production requests. This is the earliest warning of quality drift.
Retrieval health	Rising rates of low-similarity results usually mean queries have moved away from your corpus, or the index is stale.
Refusal and fallback rates	A spike means something upstream broke, often silently.
Agent step distribution	A drifting tail of long trajectories signals looping or a degraded tool.
Cost per request	Trends up quietly as context and usage grow. Alert on it.
Latency percentiles	Median hides the experience of the users most likely to churn. Watch p95 and p99.

The feedback loop

This separates a system that improves from one that decays. Production traces surface failures, failures become eval cases, eval cases drive fixes, fixes are verified against the suite. Every incident should end with a new test.

Deploying changes safely

Treat prompts, retrieval configuration, and model selection as versioned artifacts under source control with review. Roll out gradually: shadow traffic, a small percentage, then full. Keep rollback fast and rehearsed. A prompt change is a production deploy and can break just as much as a code change.

BUILD MILESTONE

Full tracing on your system, with a dashboard showing quality, cost, and latency over time. Then run a game day: introduce a subtle quality regression and see whether your monitoring catches it before you tell it to.

Security, safety, and governance

The threat model for an agent is different from a web app, because the agent acts on instructions it reads from data.

Prompt injection

The defining security problem of agentic systems, and it has no complete solution. Any content the agent reads, a web page, an email, a document, a tool response, may contain text crafted to redirect its behaviour. The agent cannot reliably distinguish your user's instructions from instructions embedded in retrieved data.

Defence is layered, and none of the layers is sufficient alone:

- **Treat all retrieved and tool content as untrusted data.** Mark it clearly as such in context. Instruct the model to never follow instructions found within it.
- **Constrain capability, not just behaviour.** The durable protection is that the agent cannot perform a damaging action, not that it was told not to. Least-privilege tool scopes.
- **Gate consequential actions** behind confirmation, allowlists, or human approval, especially anything that sends data outward.
- **Watch exfiltration paths.** An agent that reads private data and makes outbound requests can be induced to leak. Restrict outbound destinations.

UNDERSTAND THE LIMITS

Prompt-level defences reduce risk; they do not eliminate it. Architect on the assumption that injection will sometimes succeed, and make the blast radius small enough to survive.

Data boundaries and access control

Retrieval must respect the requesting user's permissions. Filtering after retrieval is not enough if retrieval itself leaks existence or content. Enforce permissions at the index and query level. In multi-tenant systems, isolate at the data layer and test that isolation adversarially.

Decide explicitly and in writing: what user data enters prompts, where it is logged, how long traces with personal data are retained, and whether any provider retains or trains on it. Answer these before launch, not during an audit.

Output safety

- Never render model output as trusted markup or execute it as code unsanitized. Standard injection risks apply.
- Filter for the harm categories relevant to your domain, and log what you filter.
- For high-stakes domains, require citations and make unsupported claims visible rather than fluent.

Governance

What a mature team can produce on request: an inventory of AI features and their risk level, documented human-oversight points, retained evaluation results, incident history, and a named owner. Regulatory expectations are still developing and vary by jurisdiction and sector, so verify current requirements for yours.

BUILD MILESTONE

Write a threat model for your system: what an attacker would want, what they could reach, and what stops them. Then attempt injection against your own agent through every content path it reads. Document what worked.

Practising this beyond the checklist

Skills compound differently once you are building for other people.

A twelve-week shape

WEEKS	FOCUS	OUTPUT
1 to 2	Evaluation	Eval dataset, grader suite, CI integration, baseline score
3 to 4	Reliability	Failure handling for every dependency, chaos tests in the suite
5 to 7	Architecture	Context management, memory design, justified topology
8 to 9	Serving and cost	Streaming API, caching, routing, cost and latency profile
10 to 11	Observability	Tracing, dashboards, alerts, production feedback loop
12	Security review	Threat model, injection testing, permission audit

Habits worth building

- **Read provider documentation directly.** More current and accurate than tutorials, and where behavioural changes are announced first.
- **Keep a failure journal.** Every production surprise, its cause, what you changed. This becomes the most valuable document you own.
- **Rebuild something with fewer moving parts.** Remove a framework layer from a working system. Understanding what the abstraction did is worth the afternoon.
- **Read other people's postmortems.** Architecture write-ups from teams running at scale teach failure modes you have not reached yet.
- **Re-run evals on new model releases.** Behaviour changes across versions in ways release notes do not fully capture.

Skills that transfer regardless of tooling

Technical

Eval design Distributed systems
 Context engineering Observability
 Cost modelling Threat modelling
 Data pipelines

Judgement

When not to use an agent
 Scoping to a measurable problem
 Reading a trace Choosing what to measure
 Explaining limits honestly

THE SHORT VERSION

Build the ruler first. Assume everything fails. Guard the context. Watch the cost. Trace everything. Constrain what the agent can actually do. The rest is iteration on one real system with real users, which is the only thing that has ever taught this well.

This roadmap describes practices and architectural patterns, not guaranteed outcomes. Model capabilities, tooling, pricing, and regulatory requirements change frequently. Verify current provider documentation and applicable regulations for your jurisdiction and sector before making production decisions.